Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Asıl ders yaklaşık 00:01:33'te başlar.
- Önceki aşamaların tekrarı
  - Ön inceleme ve proje başlatma belgesi (project charter).
  - Fizibilite çalışması ve fizibilite raporu.
  - Sistem analizi: iş analizi ve işlevsel ya da nesneye yönelik çözümleme. Çıktı: yazılım gereksinim belirtimi (SRS).
- Sistem tasarımı aşamasına giriş
  - Analiz "ne yapılmalı?" sorusunu yanıtlar. Tasarım "nasıl yapılmalı?" sorusunu yanıtlar.
  - Tasarım iki ana safhadan oluşur: ön (genel) tasarım ve ayrıntılı tasarım.
- Sistem tasarımı bileşenleri
  - Kullanıcı arayüzü tasarımı, sistem arayüzü tasarımı, güvenlik ve kontrol tasarımı, uygulama tasarımı, veritabanı tasarımı.
- Sistem tasarımı yaklaşımları
  - Model güdümlü yaklaşım (model-driven), hızlı uygulama geliştirme (RAD), ortak uygulama geliştirme (JAD).
- Ön tasarım
  - Altyapının belirlenmesi: kurum içi geliştirme, satın alıp özelleştirme, dış kaynak kullanımı (outsourcing), hizmet olarak alma.
  - İşlevsel olmayan gereksinimlerin mimariye etkisi.
  - Mimari tasarım bileşenleri: donanım (istemci, sunucu, ağ) ve yazılım (sunum, uygulama, veri katmanları).
  - Envanter, iş yükü tahmini, donanım ve yazılım değerlendirmesi, satıcı seçimi.
  - Edinme biçimi: satın alma, finansal kiralama (leasing), kiralama.
- Ayrıntılı tasarım
  - Çıktı tasarımı, girdi tasarımı, ara birim (kullanıcı arayüzü) tasarımı.
  - Yazılım (program) tasarımı ve veritabanı tasarımı. Veritabanı tasarımı ayrı bir konu olarak sonraya bırakıldı.
- İşlevsel çözümlemede yazılım tasarımı
  - Yapı diyagramı (structure chart): modüller, veri ve kontrol mesajları, koşul ve döngü gösterimleri.
  - HIPO ve IPO kartları.
- Sonraki ders: nesneye yönelik tasarım modeli.

Hocanın Özellikle Vurguladığı Kısımlar

- Bir sistemi tasarlamak yalnızca dil ve kütüphane seçmek değildir.
  - Erişim uçları, uygulama, veri, güvenlik ve diğer sistemlerle haberleşme ayrı ayrı tasarlanmalıdır.
  - Bunların hepsi tamamlanınca bilgi sistemi tasarlanmış sayılır.
- Mimari tasarımı en çok işlevsel olmayan gereksinimler (non-functional requirements) etkiler.
  - Örnek: 1 milyon kayıtlık raporu 100 ms'de üretmek, saniyede 10 bin eş zamanlı öğrenciye hizmet vermek.

- Fizibilitedeki öneriler tavsiye niteliğindedir. Tasarımda bu öneriler kesinleşir ve gerçek hayata uygulanır.
- Tasarım ne kadar ayrıntılı olursa o kadar iyidir. Açık bırakılan her nokta tasarımı muğlak ve kötü yapar.
- Ön tasarımda mümkün olduğunca az iş yapın. Mevcut donanım ve yazılım işi karşılıyorsa yeni yatırım yapmayın.
- Girdilerde dört kontrol mutlaka yapılmalıdır: tamlık, biçim, aralık, tutarlılık.
  - "Kullanıcının aptal olduğunu her zaman unutmayın." Framework kontrol ediyor diye girdiyi kontrolsüz kullanmayın.
  - SQL injection gibi açıklar da girdi kontrolünün eksikliğinden doğar.
- İyi girdi tasarımı kullanım kolaylığı sağlar.
  - Kötü örnek: silmek için menü aç, seç, işaretle, onayla.
  - İyi örnek: mobil arayüzde satırı sağa kaydırınca silme butonu çıkması.
- Yapı diyagramı ve IPO kartları bugün pek kullanılmıyor.
  - İşlevsel çözümlemede çoğu zaman fiziksel veri akış diyagramları yeterlidir.
  - Günümüzde genellikle nesneye yönelik tasarım yapılır.

Kısa Tekrar Notları

- Tasarım = "nasıl?" sorusunun cevabı.
- Ön tasarım: altyapının belirlenmesi + modüler çözümleme (yapı diyagramı). Çıktı: ön tasarım raporu (belirtimi).
- Ayrıntılı tasarım: girdi, çıktı, ara birim, yazılım ve veri tasarımı. Çıktı: ayrıntılı tasarım belirtimi.
- Her aşamanın sonunda idari inceleme yapılır. Gerekirse revize edilir.
- Tasarım yaklaşımları:
  - Model güdümlü: aday çözümler üretilir, en uygunu seçilir. Garantili ama uzun ve iş yükü fazla.
  - RAD: prototip hazırlanır, her iterasyonda geliştirilir. Spiral modele benzer.
  - JAD: bütün paydaşlar atölye çalışmasında birlikte tasarlar. Çok büyük projelerde kullanılır.
- Yazılım katmanları: sunum katmanı, uygulama (iş mantığı) katmanı, veri erişim ve veri katmanı.
- Edinme biçimleri:
  - Satın alma: uzun vadede en ucuz, tam kontrol, amortisman avantajı. Peşin yük, eskime ve yatırımın boşa gitme riski var.
  - Leasing: peşin para gerekmez. Satın almadan pahalı, kiralamadan ucuz.
  - Kiralama: ürün sizin olmaz, toplamda en pahalı. Bakım ve garanti genelde dahil.
- Girdi kontrolleri: tamlık, biçim, aralık, tutarlılık.
- Ekran düzeni: üstte veya solda gezinim alanı, ortada çalışma alanı, altta durum çubuğu.
- Yapı diyagramı sembolleri:
  - Dikdörtgen: modül. Üstteki kontrol modülü, alttakiler alt modüller.
  - Çift kenarlı dikdörtgen: kütüphane modülü.
  - İçi boş daireli ok: veri (data couple).
  - İçi dolu daireli ok: kontrol mesajı (control flag), ör. dosya sonu.
  - Eşkenar dörtgen: koşula bağlı çağrı.
  - Kavisli ok: döngü, alt modüller tekrar tekrar çalışır.
- HIPO: görsel içerik tablosu + genel IPO diyagramları + ayrıntılı IPO diyagramları.

Detaylı Açıklamalar

Analizden tasarıma geçiş

Analizde problem ve gereksinimler ortaya çıkarıldı. İş akışlarının bilgi sistemine nasıl aktarılacağı belirlendi. Tasarımda artık çözümün nasıl olacağı yazılır. Kodlanacak sistemin yapısı belgelenir. Ortaya çıkan çözüm önerisine tasarım modeli denir.

Sistem tasarımı bileşenleri

Slayttaki şekilde bir bilgi sisteminin standart parçaları var:

- **Uygulama sunucusu:** yazılımın, yani iş yapan programların durduğu yer. Uygulama tasarımı bunu ilgilendirir.
- **Veritabanı sunucusu:** verilerin tutulduğu yer. Veritabanı tasarımı bunu ilgilendirir.
- **Ofis bilgisayarları ve diğer uç noktalar:** kullanıcıların erişim noktaları. Ekranlar, raporlar, girdi ve çıktı kontrolleri kullanıcı arayüzü tasarımının konusudur.
- **Başka sistemler:** sistem başka sistemlerle veri alışverişi yapıyorsa sistem arayüzü tasarımı gerekir.
  - Örnek: Windows üzerinde IIS ile çalışan bir uygulamanın, Linux ve Java ile yazılmış MERNİS sistemine kimlik numarası sorgulatması. İki taraf belirli protokollerle konuşmalıdır.
- **Güvenlik duvarı:** parçalar internet üzerinden haberleşiyorsa sistem korunmalıdır. Güvenlik tasarımı uygulama, ağ, erişim, kullanıcı ve veri güvenliğini kapsar.

Uygulama sunucusu ile veritabanı sunucusu aynı fiziksel makinede de durabilir. Ama ayrı yazılım sunucuları oldukları için genelde ayrı makineler olarak gösterilir. Şekildeki çizim tek doğru değildir. Parçalar internetin farklı yerlerinde de olabilir.

Tasarımın aşamaları

1. Gereksinim analizinden sistem gereksinimleri gelir.
2. Ön tasarımda modüler yapı, genel kısıtlar, temel veriler ve kontrol mesajları belirlenir. Ön tasarım raporu yazılır.
3. Rapor idari incelemeden geçer. Gerekirse revize edilir.
4. Kabul edilirse ayrıntılı tasarıma geçilir. Her modül, arayüzler, veritabanı ve güvenlik tasarlanır.
5. Ayrıntılı tasarım da incelenir. Kabul edilince çözüm gerçekleştirme aşamasına aktarılır.

Sistem tasarımı yaklaşımları

- **Model güdümlü (model-driven):** Analiz modelinden birkaç aday tasarım üretilir. Artıları ve eksileri tartışılır. Gereksinimleri ve kalite beklentilerini en iyi karşılayan aday seçilir ve ayrıntılı tasarlanır. Alt türleri: modern yapısal tasarım, bilgi mühendisliği, prototipleme. Çok garantili bir ürün gerekiyorsa tercih edilir. Ama birçok adayın ön tasarımı yapılacağı için iş yükü fazladır.
- **Hızlı uygulama geliştirme (RAD):** Yaygın bir mimariden başlanır ve bir prototip hazırlanır. Paydaşlarla oturumlar yapılır. Her iterasyonda yeni modüller ve özellikler eklenir. Örnek sıra: önce çekirdek uygulama, sonra veri katmanı, sonra arayüz katmanı. Günümüzde en çok RAD ve türevleri kullanılır.
- **Ortak uygulama geliştirme (JAD):** Bütün paydaşlar birkaç gün sürebilen bir atölyede bir araya gelir. Altyapı, modüller, girdiler, çıktılar, yazılım, veri ve arayüzler birlikte tartışılır. Parça parça yapılan tasarımın sistemin başka kısımlarını bozma riskini azaltır. Çok büyük projelerde kullanılır.

Ön tasarım: altyapının belirlenmesi

Fizibiliteden gelen öneriler değerlendirilir. Yazılım, donanım ve ağ altyapısı için seçenekler:

Seçenek	Artısı	Eksisi
Kurum içi geliştirme	Tamamen size özel, fikri mülkiyet sizde, gerekirse pazarlanabilir	Pahalıya mal olur, uzun sürer, ek istihdam gerekebilir
Satın alıp özelleştirme	Toplam maliyet daha düşük olabilir	Fikri haklar sizde değil, ihtiyacı tam karşılamayabilir
Dış kaynak (outsourcing)	Hızlı, size özel çözüme yakın	Genelde kurum içi geliştirmeden bile pahalı, haklar paylaşılır
Hizmet olarak alma	İlk yatırım yok, altyapı ve geliştirmeyle uğraşılmaz	Kira gibi; zamanla toplamda pahalıya gelir

Hoca derste satın alıp hazır kullanmaktan çok, kurum içi geliştirme ve dış kaynak üzerinde durdu.

İşlevsel olmayan gereksinimler ve mimari kararlar

Use case'lerde yazmayan ama altlarındaki özel teknolojik beklentilerde geçen gereksinimler mimariyi en çok etkiler. Örnekler: rapor üretim süresi, kapasite, iki aşamalı doğrulama, zararlı yazılıma karşı koruma.

Yanıtlanması gereken mimari sorular:

- Sistem merkezi mi olacak, dağıtık mı? Yük fazlaysa merkezi sistemle karşılamak zordur. Dağıtık tasarımda eş zamanlılık sorunları da çözülmelidir.
- Veri tek merkezde mi, dağıtık veya çok kopyalı mı tutulacak? Bir kayıt güncellenince bütün kopyalar güncellenecek mi?
- Programlama dili, kütüphaneler, araçlar ne olacak?
- Hazır alınan parçalar (ör. muhasebe sistemi) nasıl entegre edilecek?
- Arayüz teknolojileri ne olacak? Mobil (iOS, Android, native veya cross-platform), web, masaüstü?
- Diğer sistemlerle hangi protokol ve güvenlik yöntemleriyle haberleşilecek?

Fizibilitede yapılan trafik ve kullanıcı sayısı projeksiyonları (ör. 10.000 yerine 100.000 kullanıcı, 5 yıllık projeksiyon) burada kesinleştirilir.

Mimari tasarım bileşenleri

- **Donanım:** istemci (akıllı telefon, saat, Kindle, sensör, kamera…), sunucu (merkezi veya dağıtık) ve ağ altyapısı.
- **Yazılım:** katmanlara ayrılır. Her katman bir alttakinden hizmet alır, bir üsttekine hizmet verir.
  - Sunum katmanı: istemcilerle ve diğer sistemlerle haberleşen kontrol ara tabakası ve ön yüzler.
  - Uygulama katmanı: uygulama mantığı.
  - Veri erişim mantığı ve veri depolama.

Envanter, iş yükü ve satıcı seçimi

1. **Envanter:** Elde ne var? Model, üretici, yaş, kalan ömür, fiziksel yer, sorumlu kişi, işletmenin malı mı kiralık mı?
2. **İş yükü tahmini:** Mevcut kapasite yeni sistemi kaldırır mı, istenen sürede tepki verir mi? Amaç gereksiz satın almayı önlemektir.
3. **Donanım değerlendirmesi:** bir işlemin ortalama süresi, toplam kapasite, CPU boşta kalma oranı, bellek ihtiyacı.
4. **Yazılım değerlendirmesi:** istekleri karşılama, performans, kapasite, kullanım kolaylığı, esneklik, dokümantasyon, üretici desteği.
5. **Satıcı seçimi:** donanım ve yazılım desteği, kurulum ve eğitim hizmetleri, bakım seviyeleri. Örnek: yalnız mesai saatleri, akşamlar dahil, 7/24 destek. Destek arttıkça fiyat da artar.
6. **Edinme biçimi:** üst yönetimle birlikte karar verilir.

Edinme biçimleri

- **Satın alma:** Uzun dönemde daha ucuzdur. Ürün tamamen sizindir, ölçeklendirebilirsiniz. Amortismanla gider yazılarak vergi avantajı sağlanır. Ama para peşin çıkar. Teknoloji hızla eskir. Yanlış seçimde yatırım boşa gider. Bütün sorumluluk sizdedir.
- **Finansal kiralama (leasing):** Banka gibi bir kurum cihazı alır, siz kira öder gibi ödersiniz. Satın almadan pahalı, kiralamadan ucuzdur. Ödeme aksarsa ürün elinizden gidebilir veya daha yüksek faiz ödersiniz.
- **Kiralama:** Ürün sizin olmaz. Toplam kullanım ömründe en pahalı seçenektir. Bakım ve garanti genelde dahildir. Ek finansman gerekmez. Abonelikle aylık gelir toplayan işletmeler için uygun olabilir. Bırakınca elden çıkarılacak donanım kalmaz.

Çıktı tasarımı

Tasarımın ana sorusu yine “nasıl?” sorusudur. Bir çıktıyı tasarlarken:

- Çıktı bir amaca hizmet etmeli ve kullanıcı için anlamlı olmalı. Alakasız içerik olmamalı.
- Uygun sayıda çıktı üretilmeli.
- Hangi kullanıcıların erişeceği doğru belirlenmeli.
- Zaman ekseni varsa (günlük, aylık, yıllık) çıktı buna uygun üretilmeli.
- Doğru ortamda üretilmeli: ekran, pano, kağıt liste, akıllı saat bildirimi vb.
- Temiz ve yalın olmalı. Listeler sıralanabilmeli ve filtrelenebilmeli. Sınırlar belli olmalı.
- Grafik tipi ve renkleri doğru seçilmeli, ölçeklenebilir olmalı.

Girdi tasarımı

Amaç kaliteli girdi elde etmektir.

- Her ekran kendi içinde bütünlük taşımalı. Kişisel bilgiler ekranında eğitim bilgileri olmamalı.
- Analizde tanımlanan bütün işlemler için gerekli bütün veriler alınmalı.
- Ergonomik giriş sağlanmalı. Örnek: yıl için 4 tuş yerine 2-3 tuş yeter. 31 gün arasından kaydırma çubuğuyla seçim ergonomik değildir.
- Ekranlar arasında uyum olmalı. Fontlar ve yerleşim bir anda değişmemeli.
- Toplu (batch/bulk) girdi desteklenmeli. Örnek: notları OBS’ye tek tek girmek yerine Excel’den içe aktarmak.
- Veri mümkün olduğunca kaynağında tutulmalı.
- Tıklama ve tuşlama sayısı az olmalı.
- Doğrulama ve geçerleme (verification and validation) mutlaka yapılmalı.

Dört temel kontrol, doğum tarihi örneğiyle:

Kontrol	Örnek hata
Tamlık	Yalnız yıl (1999) girildi. Gün ve ay eksik.
Biçim	Gün-ay-yıl mı, ay-gün-yıl mı? ABD’de ay-gün-yıl alışkanlığı var.
Aralık	1899 girildi (124 yaş) veya 2122 girildi (gelecekte bir tarih).
Tutarlılık	Doğum 1999, işe giriş 2003: kişi 4 yaşında işe girmiş görünüyor.

Mobil uygulamalarda “Google ile giriş yap” gibi seçenekler de kolay girdi tasarımı örneğidir.

Ara birim (kullanıcı arayüzü) tasarımı

- Ekranlar basit olmalı, gereksiz bilgi içermemeli. Kutulama yapısı ve açıklamalar kullanılmalı.
- Ekranlar arasında tasarım uyumu ve kolay geçiş olmalı.
- Renk, yazı tipi, imleç ve boyutlar sürekli değişmemeli. Web’de CSS bu işi kolaylaştırır.
- Ekran ile birlikte başka ara birimler de tasarlanabilir: doğal dil (ChatGPT benzeri), soru-cevap, menü, form, uygulamaya özel komut dili, sistem bildirimleri.
- Tipik ekran düzeni:
  - Üstte veya solda gezinim (navigation) alanı. Örnek: Word’ün üst menüsü, OBS’nin sol menüsü.
  - Ortada en büyük alan: kullanıcının çalıştığı ana ekran.
  - Altta durum çubuğu (status bar).
- Sık görülen kusurlar: düzensiz bilgi yığınları, uyumsuz geçişler, karmaşık terminoloji, açıklamasız hata mesajları (“bilinmeyen bir hata oluştu”), anlaşılmaz kullanıcı kılavuzları.

Yapı diyagramı (structure chart)

İşlevsel çözümlemede (veri akış diyagramlarıyla) yola çıkıldıysa modüler yapı yapı diyagramıyla gösterilir. Modüler program yapısı, veri akış diyagramındaki işlevleri yazılım modüllerine dönüştürmek ve aralarındaki hiyerarşiyi, ilişkileri ve kontrolleri göstermektir.

Slayt örnekleri:

- **Maintain Customer Data:** iki alt modülü var: Check Customer Status ve Update Customer File.
  - Check Customer Status'a Customer Number (veri) gönderilir. Customer Status (veri) geri döner.
  - Update Customer File'a Customer Charges (veri) gönderilir. Account Overdue (kontrol mesajı) geri dönebilir.
  - Slaytta bu mesaj "Control Couple" olarak işaretli; hoca "kontrol çifti" dedi (01:15:38). Üst modül gelen kontrol mesajını ele almalıdır.
- **Sort Inventory Parts:** eşkenar dörtgen koşulu gösterir. Koşula göre Sort By Part Name, Sort By Part Number veya Sort By Inventory Value çalışır.
- **Produce Grade Reports:** kavisli ok, Get Student Grades ve Calculate GPA'nın tekrar tekrar çalıştığını gösterir. Get Student Grades'e Student Number gönderilir. Grades ve End of File (kontrol) döner. Calculate GPA'ya Grades gönderilir, GPA döner. Sonra Produce Grade Report'a GPA Listing gönderilir, Grade Report döner.
- **Generate Payroll:** Get Payroll Record (Read Payroll Record, Validate Payroll Record), Calculate Net Pay (Calculate Gross Pay, Calculate Deductions, Update Employee Record) ve Print Check. Calculate Deductions altında Calculate Tax Withheld ve Calculate SS Withheld var. Ödeme belgesi basılınca Check Printed kontrol mesajı döner.

Yapı diyagramı ön tasarımda modüllerin iç algoritmasını göstermez. Yalnız modülün çalışması için gerekenleri ve ürettiği çıktıyı gösterir. Veri akış diyagramındaki 1.1, 1.2 gibi ikinci seviye işlemler yapı diyagramında alt modüllere karşılık gelir.

HIPO ve IPO kartları

- **IPO (Input-Process-Output) kartı:** her modül için girdiler, işlem ve çıktılar yazılır.
- **HIPO (Hierarchy + IPO):** modüllerin hiyerarşisi ve IPO kartları birlikte verilir.
  - Görsel içerik tablosu: kitabın içindekiler tablosu gibi, ana menü ve alt menüler.
  - Genel IPO diyagramları ve ayrıntılı IPO diyagramları.
  - Ayrıntılı diyagramlarda adımlar sözde kod (pseudocode) veya akış diyagramıyla yazılabilir.
- Hocaya göre IPO tarafı pratikte yapılmaz. Modüler çözümleme için yapı diyagramı, çoğu zaman fiziksel veri akış diyagramı yeterlidir.

Ders sonrası notlar

- **"DFT":** Altyazıda veri akış diyagramları için "DFT'lerimiz" geçiyor. Doğru kısaltma DFD'dir (Data Flow Diagram).
- **"IPO = Hierarchy, input, process, output":** Altyazıda "Ne demektir IPO? Hierarchy yani input, process, output" geçiyor (01:23:46). Hoca burada HIPO demiş, altyazı bozulmuş olabilir. Hiyerarşiyi de içeren yöntemin adı HIPO'dur (Hierarchy plus Input-Process-Output). IPO yalnızca Input-Process-Output demektir.
- **Account Overdue:** Hoca bu kontrol mesajını "hesap bakiyesi yetersiz" ya da "hesap taştı" diye açıkladı. "Overdue" vadesi geçmiş, gecikmiş ödeme demektir. Yani mesaj, müşterinin vadesi geçmiş borcu olduğunu bildirir.
- **"Kontrol çifti":** Yapı diyagramı terminolojisinde "couple", modüller arasında taşınan öğedir. Control couple kontrol mesajının kendisidir (control flag); veri ile kontrolün çifti demek değildir.
- **Veri ve kontrol okları:** Yapı diyagramında içi boş daireli ok veri (data couple), içi dolu daireli ok kontrol mesajıdır (control flag). Slaytlarda da bu ayrım böyle; altyazıda yalnız "içi boş halka" açıkça söylendi.
- **Satın almada vergi avantajı:** Derste amortismanın "gelir vergisinden" düşüldüğü söylendi. Şirketlerde amortisman gider olarak yazılır ve vergi matrahını azaltır. Şirketin türüne göre bu kurumlar vergisi de olabilir.

- Altyazıda bazı kelimeler bozuk geçiyor: “DRV”, “slide map”, “gülümlü” (güdümlü), “A altyapısı” (ağ altyapısı), “sonucu” (sunucu). Özette doğru hâlleri kullanıldı.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi daha verimli çalışabilirsiniz.